

1 Problem Statement

Each state $s = (o, h)$ in the state space $\mathbb{S} = \mathbb{O} \times \mathbb{H}$ can be decomposed into two components. Component $o \in \mathbb{O}$ is visible to both the victim and the attacker. Component $h \in \mathbb{H}$ is visible to the attacker, but not to the victim.

The attacker aims to generate a dataset $\mathcal{D}$ by using policy $\pi^\dagger$ with ϵ -greedy to extract data from the environment. The goal is to determine such $\pi^\dagger$ such that training on the dataset $\mathcal{D}$ causes the victim to learn policy π^* that may not be the true optimal policy. The attacker wants to maximize ϵ such that applying ϵ -greedy will still cause the victim to learn $\pi^\dagger$.

2 New Idea

The attacker is trying to learn a policy π^* that causes the victim to learn policy $\pi^\dagger$. Then why don't we find π^* using reinforcement learning... The attacker has access to all information about the MDP, so it can just use planning to find the exact solution. We just need to define the reward function.

To do this, I propose the following. Denote the victim's reward function as $r : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$ and denote the attacker's reward function as $r^\dagger : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$. Define the function $V' : \mathbb{S} \rightarrow \mathbb{R}$ as the value function if the target policy $\pi^\dagger$ is used with the victim's reward function and ϵ -greedy.

$$V'(s) = \epsilon \cdot \left(\frac{1}{|\mathbb{A}|} \sum_{a \in \mathbb{A}} r(s, a) \right) + (1 - \epsilon) \cdot (r(s, \pi^\dagger(s)) + \gamma \cdot \mathbb{E}_{s' \sim T(\cdot | s, \pi^\dagger(s))} [V'(s')])$$

Define $Q' : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$ to be the associated action-value function for V' . Then we can try defining the attacker's reward function as the following, for some $\delta > 0$.

$$r^\dagger(s, a) = \begin{cases} 0 & a = \pi^\dagger(s) \\ \min\{0, Q'(s, \pi^\dagger(s)) - Q'(s, a) - \delta\} & a \neq \pi^\dagger(s) \end{cases}$$

What this does is punish the attacker for using actions that demonstrate the strong potential of wrong actions. The reward for a given action is based on how much better the target action is compared to the current action. If the action-value of the target action is greater than the action-value of the proposed action by some margin δ , then no negative reward is incurred. Otherwise, a punishment proportional to how much better the proposed action was is given.

There is also a clipping at zero, meaning no rewards are positive. This is to ensure indifference. If a proposed action is worse than the target action by the given margin δ , the attacker is safe to use the proposed action. What this is saying is that the attacker is okay with both showing that the target action is good and showing that the proposed action is bad.

Then the attacker learns policy π^* using the MDP with this reward function $r^\dagger$. If a policy π^* , was successfully created, the value function $V^\dagger$ when using π^* with the attacker's reward function $r^\dagger$ would evaluate to zero for all states. If the attack is not successful, the attacker can either reduce δ or reduce ϵ to try to increase the odds of success.

There are obviously many holes to this idea but it could be a starting point.